

OOPS

Practical File

Submitted by:-

Kaustubh Sharma

02451203119

IT 2nd Shift

INDEX

S.No.	Date	Experiment	Remarks
1	5/4/21	Write a program for multiplication of two matrices using OOP.	
2	12/4/21	Write a program to perform addition of two complex numbers using constructor overloading.	
3	19/4/21	Write a program to find the greater of two given numbers in two different classes using friend function.	
4	17/5/21	Write a program to implement a class string which uses operator overloading to perform operations on strings	
5	24/5/21	Write a program to define the function template for calculating the square of given numbers with different data types.	
6	31/5/21	Write a program to demonstrate the use of special functions, constructor and destructor in the class template. The program is used to find the bigger of two entered numbers.	
7	7/6/21	Write a program to perform the deletion of white spaces such as horizontal tab, vertical tab, space, line feed, new line and carriage return from a text file and store the contents of the file without the white spaces on another file.	
8	14/6/21	Write a program to read the class object of student info such as name , age ,sex ,height and weight from the keyboard and to store them on a specified file using read() and write() functions. Again the same file is opened for reading and displaying the contents of the file on the screen.	
9		CBS1:- You are given two unmarked jugs with capacities x and y liters. Determine the moves to obtain exactly n liters of water in any of the two jugs or both by the end	
10		CBS2:- WAP to make Tic Tac Toe game.	

Experiment 1

Aim:- WAP for multiplication of two matrices using OOP.

Theory:-

The object oriented programming paradigm is a superset of object based programming. It offers all the features of object based programming and overcome its limitation by implementing inheritance so that the world relations among objects can be represented programmatically.

OOP refers to a type of computer programming in which programmers define the data type of data structure and also the type of operations that can be applied to the data structures. It enables us to create modules that do not need to be changed when new object is added.

Source Code

```
#include <iostream.h>
class matrix
{
    int a[3][3];
    // int b[3][3];
    // int c[3][3];
    int i, j, k;
public:
    matrix mult();
}
```

GOOD WRITE

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

30	36	42
----	----	----

66	81	96
----	----	----

102	126	150
-----	-----	-----

```

void input();
void output();
};

matrix::input()
{
    for (i=0; i<3; i++)
        for (j=0; j<3; j++)
            cin >> a[i][j];
}

matrix::mult (matrix f, matrix g)
{
    matrix c;
    for (i=0; i<3; i++)
        for (j=0; j<3; j++)
        {
            c.a[i][j]=0;
            for (k=0; k<3; k++)
                c.a[i][j] += f.a[i][k] * g.a[k][j];
        }
    return c;
}

void matrix::output()
{
    for (i=0; i<3; i++)
        for (j=0; j<3; j++)
            cout << a[i][j];
}

void main()
{
    matrix a, b, c, d;
    a.input(); b.input();
    c = d.mult(a, b);
    c.output();
}

```

Experiment 2

Aim:- WAP to perform addition of two complex numbers using constructor overloading.

Theory:-

Constructor Overloading:- In C++, we can have more than one constructor in a class with same name, as long as each has different list of arguments. It is quite similar to function overloading. Overloaded constructor essentially have the same and different ~~no~~ number of arguments. A constructor is called depending upon the number and type of arguments passed. While creating object, arguments must be passed, ~~while to~~ ~~had~~ let compiler know which constructor to call.

Source Code

```
#include <iostream.h>
class complex
{ int real, imag;
  public
    complex()                // Default
    { }
    complex(int r)           // Parameterised same value
    { real: r;
      imag: r;
    }
}
```

GOOD WRITE

Enter Value 1: 2

Enter Value 2: 3

Answer: $4 + i5$

```

complex (int r, int i)           // parameterised different values
{
    real = r;
    imag = i;
}

void output ()
{
    cout << "Answer:" << real << "+i" << imag;
}

friend complex sum (complex, complex);

complex sum (complex c1, complex c2)
{
    complex c3;
    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;
    return c3;
}

void main ()
{
    complex c1;           // Default
    int a, b, c;
    cout << "Enter value 1:"; cin >> a;
    cout << "Enter value 2:"; cin >> b;
    complex c2(a);
    complex c3(a, b);
    complex c4: sum(c2, c3);
    c4.output();
}

```

Experiment 3

Aim:- WAP to find greatest of two numbers in two different classes using friend function.

Theory:-

Friend Function:- A friend of a class X is a function of class, although, not a member of class, has full access rights to the private and ~~protected~~ protected members of class X.

In all other respects, friend is a normal function in terms of scope, declaration and definitions. It is sometimes useful to allow particular class to access private members of other class.

Source Code

```
#include <iostream.h>
class second;
class first
{ int a;
  public:
  first(int b)
  { a = b; }
  friend int max(first, second);
}
class second
{ int c;
```

GOOD WRITE

Enter 2 Numbers : 2 3
Greater Number: 3

```

public:
    second (int d)
    { c = d ; }
    friend int max (first, second);
}

int max ( first a, second b)
{ return (a.c > b.c) ? a.c : b.c;
}

void main()
{ int a, b;
  cout << "enter 2 number numbers:";
  cin >> a >> b;
  first f(a); second s(b);
  cout << "the greater is: " << max(f, s);
  getch();
}

```

Experiment 4

Aim:- WAP to implement a class string which uses operator overloading to perform operations on strings.

Theory

Operations overloading is a compile time polymorphism in which the operator is overloaded to provide the special meaning to the user defined data types. For ex; C++ provides ability to add the variables of the user defined data types that are applied to the built in data types. The advantage of operators overloading is to perform different operations on the same operand.

Existing operators can be overloaded, but the new operators cannot be overloaded. Friend function cannot be used to overload certain operators.

Source Code

```
#include <iostream.h>
#include <type.h>
#include <string.h>
class string
{ char s1[25], s2[25];
public:
    string (char str1[], char str2[])
    { strcpy (s1, str1);
```

GOOD WRITE

Enter strings : Kaustubh
Sharma

Concatination : KaustubhSharma

Equated : String 1 : Sharma
String 2 : Sharma

String 1 and 2 are equal

length : 6
sharma
SHARMA

```

strcpy (s2, str2);
}
void operator +()
{ cout << "concatination:" << strcat (s1, s2);
  cout << endl;
}
void operator =()
{ strcpy (s1, s2)
  cout << "string 1:" << s1 << "string 2:" << s2 << endl;
}
void operator <=()
{ int a = strcmp (s1, s2)
  if (a > 0)
    cout << "string 1 is greater";
  else if (a == 0)
    cout << "string 2 and is greater are equal";
  else
    cout << "string 2 is greater";
}
void length()
{ cout << "length:" << strlen (s1); }
void lower()
{ cout << tolower (s1);
}
void Upper()
{ cout << toupper (s1);
}
}
}

```

GOOD WRITE

```

void main()
{ cout << "Enter strings ";
  char s1[25], s2[25];
  cin >> s1 >> s2;
  string a(s1, s2)
  + a;
  = a;
  < = a;
  a.length();
  a.lower();
  a.upper();
}

```

Experiment 5

Aim:- WAP to define the function template for calculating square of given numbers with different data types.

Theory

Function templates are special functions that can operate with generic types. This allows to create a function template whose functionality can be adopted to more than one type or class without repeating the entire code for each type.

When the compiler encounters call to a function template, it uses the template to automatically generate a function replacing each appearance of type by the actual template parameters.

Source Code

```
#include <iostream.h>
template <typename T>
T square (T x)
{ return x * x;
}

void main ()
{ int a; float b;
  double c;
  cout << "Enter integers:";
  cin >> a;
```

GOOD WRITE

Enter integer value : 12

Enter float value : 1.2

Enter double value : 2.34

Square of integer : 144

Square of float : 1.44

Square of double : 5.4756

```
cout << "Enter float: ";  
cin >> b;  
cout << "Enter double";  
cin >> c;  
cout << square < int > (a) << endl;  
cout << square < float > (b) << endl;  
cout << square < double > (c) << endl;
```

}

Experiment 6

Aim:- WAP to demonstrate use of special functions, constructor and destructor in the class template. The program is used to find bigger of two entered numbers.

Theory

A class template produces a specification for generating classes based on parameters. Class templates are generally used to implement by passing a given set of types to it as template argument.

Like function template, class templates are useful when a class defines something that is independent of the data types. The template allows us to define generic classes. The prefix tells compiler that we are defining a template.

Source Code

```
#include <iostream.h>
```

```
template < class T>
```

```
class Largest
```

```
{ private:
```

```
    T num1, num2;
```

```
    public:
```

```
    Largest(T n1, T n2)
```

```
{ m num1 = n1;
```

```
    num2 = n2; }
```

GOOD WRITE

Integer Values : 2 1

Float Values : 3.2 5.6

Integer : 2

Float : 5.6

```
void display()
{ cout << "larger number : " << add() << endl;
  }
  T add()
  { return (num1 > num2) ? num1 : num2;
    }
  };
```

```
int main()
{ largest < int > int large (2, 1);
  largest < float > float large (3.2, 5.6);
  cout << "Integer : " << endl;
  int large . display();
  cout << "Float : " << endl;
  float large : display();
}
```

Experiment 7

Aim:- WAP to perform deletion of white spaces from a text file and store the result on another file.

Theory

File represent storage medium for string data or information stream refers to sequence of bytes. In files; we store data i.e text or binary data permanently and use the data to read and write ~~int~~ in the form of input output operations by transferring bytes of data. File operation are contained in header file `fstream.h`. `ofstream` is used to represent output stream and is used for writing in file. `ifstream` refers input file stream. It can read from file.

Source Code

```
#include <fstream.h>
#include <iomanip.h>
#include <conio.h>
void main()
{ clrscr();
  ofstream outfile;
  ifstream infile;
  char fname fname1[10], fname2[10];
  char ch;
  cout << "Enter file to be copied: ";
```

GOOD WRITE

Enter file to be copied : test.txt

Enter new file : out.txt

Contents of file 1

Hello,

I am Kaustubh Sharma

Contents of file 2

Hello, I am Kaustubh Sharma

```

cin >> fname1;
cout << "Enter new file";
cin >> fname2;
infile.open(fname1);
outfile.open(fname2);
while (!infile.eof())
{
    char = (char)infile.get();
    if (char == '|' || char == '\\t' || ch == '\\n');
    else
        outfile.put(ch);
}
cout << "contents of file 1";
while (!infile.eof())
{
    ch = (char)infile.get();
    cout << ch;
}
cout << "contents of file 2";
while (!outfile.eof())
{
    cout << (char)outfile.get();
}
infile.close();
outfile.close();
}

```

Experiment 8

Aim:- WAP to read class object of student info from keyboard and store them on a specified file. Again the same file is opened for reading and displaying the contents.

Theory

Filestream includes two member functions specifically designed to read and write binary data ~~subsequently~~ sequentially. Write is a member of ofstream and read is a member of ifstream. The functions read() and write() handle the data in binary form. This means that values are stored in the data file in the same formation in which they are stored in internal memory. These functions take two arguments. First is the address second is the length.

Source Code

```
#include <fstream.h>
class student_info {
    char name[20];
    int age;
    char sex;
    float height, weights;
public:
    void input();
```

GOOD WRITE

Writing to file
Enter name : xyz
Enter age : 20
Enter sex : M
Enter height : 180

Reading from file
Name : xyz
Age : 20
Sex : M
Height : 180

```

{ cout << "Enter name "; gets (name);
  cout << "Enter age "; cin >> age;
  cout << "Enter sex "; cin >> sex;
  cout << "Enter height "; cin >> height;
}

void output ()
{ cout << "Name "; puts (name);
  cout << "Age " << age;
  cout << "sex " << sex << endl;
  cout << "Height " << height << endl;
}

void main ()
{ clrscr ();
  student info s1, s2;
  fstream outfile;
  outfile.open ("p31.dat", ios::out | ios::binary);
  cout << "Writing to file" << endl;
  s1.input ();
  outfile.write ((char *) &s1, sizeof(s1));
  outfile.close ();
  fstream infile;
  infile.open ("p31.dat", ios::in | ios::binary);
  while (infile.read ((char *) &s2, sizeof(s2)));
  { s2.output (); }
  infile.close ();
  getch ();
}

```

Content Beyond Syllabus 1

Aim:- You are given two unmarked jugs with capacities x and y liters. Determine the moves to obtain exactly n liters of water in any of the two jugs or both by the end.

Given that:

1. There is infinite supply of water.
2. Both the jugs are empty at the beginning.

Operations allowed:

1. Empty /fill a jug completely with water.
2. Pour water from one jug to another until one of the jugs is either empty or full.

Source Code:-

```
#include<bits/stdc++.h>
using namespace std;
int x;
int y;
void show(int a, int b);
int min(int w, int z)
{
    if (w < z)
        return w;
    else
        return z;
}
void show(int a, int b)
{
    cout << setw(12) << a << setw(12) << b<<endl;
}
void s(int n)
{
    int xq = 0, yq = 0;
    int t;
    cout << setw(15) <<"FIRST JUG"<< setw(15) <<"SECOND JUG"<<endl;
    while (xq != n && yq!=n )
    {
```

```

if (xq == 0)
{
    xq = x;
    show(xq, yq);
}
else if (yq == y)
{
    yq = 0;
    show(xq, yq);
}
else
{
    t = min(y - yq, xq);
    yq = yq + t;
    xq = xq - t;
    show(xq, yq);
}
}
}
int main()
{
    int n;
    cout << "Enter the liters of water required out of the two jugs: ";
    cin >> n;
    cout << "Enter the capacity of the first jug: ";
    cin >> x;
    cout << "Enter the capacity of the second jug: ";
    cin >> y;
    if(n < x || n < y)
    { if(n % (__gcd(x, y)) == 0)
        s(n);
        else
        cout << "This is not possible....\n";
    }
    else
        cout << "This is not possible....\n";
}

```

Output:-

```
Enter the liters of water required out of the two jugs: 2
```

```
Enter the capacity of the first jug: 4
```

```
Enter the capacity of the second jug: 3
```

FIRST JUG	SECOND JUG
4	0
1	3
1	0
0	1
4	1
2	3

Content Beyond Syllabus 2

Aim:- WAP to make Tic Tac Toe game.

Source Code:-

```
#include <iostream>
using namespace std;

char square[10] = {'o','1','2','3','4','5','6','7','8','9'};
int checkwin();
void board();
int main()
{
    int player = 1,i,choice;
    char mark;
    do
    {
        board();
        player=(player%2)?1:2;
        cout << "Player " << player << ", enter a number: ";
        cin >> choice;
        mark=(player == 1) ? 'X' : 'O';
        if (choice == 1 && square[1] == '1')
            square[1] = mark;
        else if (choice == 2 && square[2] == '2')
            square[2] = mark;
        else if (choice == 3 && square[3] == '3')
            square[3] = mark;
        else if (choice == 4 && square[4] == '4')
            square[4] = mark;
        else if (choice == 5 && square[5] == '5')
            square[5] = mark;
        else if (choice == 6 && square[6] == '6')
            square[6] = mark;
        else if (choice == 7 && square[7] == '7')
            square[7] = mark;
        else if (choice == 8 && square[8] == '8')
            square[8] = mark;
```

```

else if (choice == 9 && square[9] == '9')
    square[9] = mark;
else
{
    cout<<"Invalid move ";
    player--;
    cin.ignore();
    cin.get();
}
i=checkwin();
player++;
}while(i!=-1);
board();
if(i==1)
    cout<<"==>\aPlayer "<<--player<<" win ";
else
    cout<<"==>\aGame draw";
cin.ignore();
cin.get();
return 0;
}
/*****
FUNCTION TO RETURN GAME STATUS
1 FOR GAME IS OVER WITH RESULT
-1 FOR GAME IS IN PROGRESS
0 GAME IS OVER AND NO RESULT
*****/
int checkwin()
{
    if (square[1] == square[2] && square[2] == square[3])
        return 1;
    else if (square[4] == square[5] && square[5] == square[6])
        return 1;
    else if (square[7] == square[8] && square[8] == square[9])
        return 1;
    else if (square[1] == square[4] && square[4] == square[7])
        return 1;
    else if (square[2] == square[5] && square[5] == square[8])
        return 1;
    else if (square[3] == square[6] && square[6] == square[9])

```

```

        return 1;
    else if (square[1] == square[5] && square[5] == square[9])
        return 1;
    else if (square[3] == square[5] && square[5] == square[7])
        return 1;
    else if (square[1] != '1' && square[2] != '2' && square[3] != '3'
        && square[4] != '4' && square[5] != '5' && square[6] != '6'
        && square[7] != '7' && square[8] != '8' && square[9] != '9')
        return 0;
    else
        return -1;
}
/*****
*

FUNCTION TO DRAW BOARD OF TIC TAC TOE WITH PLAYERS
MARK
*****/
void board()
{
    system("cls");
    cout << "\n\n\tTic Tac Toe\n\n";
    cout << "Player 1 (X) - Player 2 (O)" << endl << endl;
    cout << endl;
    cout << "   |   |   " << endl;
    cout << " " << square[1] << " | " << square[2] << " | " << square[3] << endl;
    cout << "_____|_____|_____" << endl;
    cout << "   |   |   " << endl;
    cout << " " << square[4] << " | " << square[5] << " | " << square[6] << endl;
    cout << "_____|_____|_____" << endl;
    cout << "   |   |   " << endl;
    cout << " " << square[7] << " | " << square[8] << " | " << square[9] << endl;
    cout << "   |   |   " << endl << endl;
}

```

Output:-

```

      Tic Tac Toe
Player 1 <X>  -  Player 2 <O>

 1 | 2 | 3
--|---|
 4 | 5 | 6
--|---|
 7 | 8 | 9

Player 1, enter a number: 1_
```

```

      Tic Tac Toe
Player 1 <X>  -  Player 2 <O>

 X | 2 | 3
--|---|
 4 | 5 | 6
--|---|
 7 | 8 | 9

Player 2, enter a number: 5_
```

```

      Tic Tac Toe
Player 1 <X>  -  Player 2 <O>

 X | 2 | 3
--|---|
 4 | 0 | 6
--|---|
 7 | 8 | 9

Player 1, enter a number: 7_
```

```

      Tic Tac Toe
Player 1 (X)  -  Player 2 (O)

  X | 2 | 3
  --+---+--
  4 | 0 | 6
  --+---+--
  X | 8 | 9

Player 2, enter a number: 4

```

```

      Tic Tac Toe
Player 1 (X)  -  Player 2 (O)

  X | 2 | 3
  --+---+--
  O | 0 | 6
  --+---+--
  X | 8 | 9

Player 1, enter a number: 3

```

```

      Tic Tac Toe
Player 1 (X)  -  Player 2 (O)

  X | 2 | X
  --+---+--
  O | 0 | 6
  --+---+--
  X | 8 | 9

Player 2, enter a number: 6

```

Tic Tac Toe

Player 1 (X) - Player 2 (O)

X	2	X
O	O	O
X	8	9

=>Player 2 win _